The Acme Editor

A Comprehensive Guide

Based on the standalone acme derived from Plan 9

The acme editor was created by Rob Pike at Bell Labs as part of the Plan 9 operating system. The standalone version documented here is derived from plan9port (Plan 9 from User Space) by Russ Cox.

Acme source code: MIT License.
Copyright © 2021 Plan 9 Foundation.
Portions copyright © 2001–2008 Russ Cox.
Portions copyright © 2008–2009 Google Inc.

Contents

Contents iii

1 Introduction 1
- 1.1 Philosophy . . . 1
- 1.2 How Acme Differs . . . 2
- 1.3 History . . . 3
- 1.4 About This Book . . . 3

2 Installation 4
- 2.1 Linux . . . 4
- 2.2 macOS . . . 5
- 2.3 Windows . . . 6
- 2.4 Command-Line Options . . . 8
- 2.5 Environment Variables . . . 8

3 The Acme Screen 10
- 3.1 Anatomy of the Display . . . 10
- 3.2 Text Everywhere . . . 12
- 3.3 Window Management . . . 13
- 3.4 Directory Windows . . . 13
- 3.5 The +Errors Window . . . 13

4 The Mouse 14
- 4.1 Button 1: Select . . . 14
- 4.2 Button 2: Execute . . . 15
- 4.3 Button 3: Look and Open . . . 16
- 4.4 Mouse Chords . . . 16
- 4.5 The Scrollbar . . . 18
- 4.6 Scroll Wheel . . . 18
- 4.7 One-Button and Two-Button Mice . . . 18

4.8 Summary 19

5 Keyboard **20**
5.1 Cursor Movement 20
5.2 Page Navigation 21
5.3 Jump to Position 21
5.4 Editing Keys 22
5.5 Text Entry 22
5.6 Unicode Composition 22
5.7 Summary 23

6 Built-in Commands **25**
6.1 Clipboard: Cut, Snarf, Paste 25
6.2 Undo and Redo 26
6.3 File Operations: Get, Put, Putall 26
6.4 Window Management 26
6.5 Session Management: Dump, Load, Exit 27
6.6 Search: Look 27
6.7 Display: Font, Tab, Indent 28
6.8 Process Control: Kill, ID 28
6.9 Include Paths: Incl 28
6.10 Send 29
6.11 Win 29

7 Shell Commands and Pipes **30**
7.1 Running External Commands 30
7.2 Pipe Prefixes: <, >, | 31
7.3 The +Errors Window 32
7.4 The Win Command 32
7.5 When to Use What 34

8 The Edit Command **35**
8.1 Addresses 35
8.2 Simple Commands 36
8.3 Structural Commands 37
8.4 Grouping 38
8.5 Multi-File Commands 38
8.6 Substitution Details 38
8.7 Worked Examples 39
8.8 What Is Not Implemented 40

8.9 Further Reading 40

9 Plumbing **41**
9.1 How Plumbing Works 41
9.2 Default Rules 42
9.3 Custom Rules 42
9.4 Debugging Plumbing 44

10 Practical Workflows **46**
10.1 Editing Code 46
10.2 Building and Testing 47
10.3 Version Control 48
10.4 Writing Prose 48
10.5 Multi-File Search and Replace 49
10.6 Session Management 50
10.7 Tips 50

11 Customization **52**
11.1 Fonts 52
11.2 Tab Width 53
11.3 Auto-Indentation 53
11.4 Plumbing Rules 54
11.5 Shell 54
11.6 Helper Scripts 55
11.7 Window Size 56
11.8 Scrollbar Direction 56
11.9 Startup Script 56

A Command Reference **58**

B Edit Command Reference **60**
B.1 Addresses 60
B.2 Commands 60
B.3 Structural Commands 61
B.4 Multi-File Commands 61
B.5 Substitute Backreferences 61

C Mouse Reference **63**
C.1 Single-Button Actions 63
C.2 Chords 63
C.3 Modifier-Click Emulation 64

C.4 Button 3 Decision Table 64

D Keyboard Reference **65**
D.1 Cursor Movement 65
D.2 Page and File Navigation 65
D.3 Editing 66
D.4 Composition 66

E Unicode Composition Table **67**
E.1 Latin Accented Characters 67
E.2 Punctuation and Symbols 68
E.3 Greek Letters 69
E.4 Mathematical Symbols 69
E.5 Arrows 70
E.6 Currency 71

F Default Plumbing Rules **72**
F.1 Variable Definitions 72
F.2 URLs 72
F.3 Image Files 73
F.4 PDF / PostScript / DVI Files 73
F.5 File Addresses with Two-Part Line:Column 73
F.6 File Addresses with Line:Column 74
F.7 Existing Files with Optional Address 74
F.8 Include Files 74

Bibliography **76**

Index **78**

CHAPTER 1

Introduction

Acme is a text editor and programming environment created by Rob Pike at Bell Labs in 1993. It was designed as part of the Plan 9 operating system, where it served as both an editor and a shell—a surface on which all work takes place.

Acme is unlike any other editor you have used. It has no menus, no toolbars, no configuration files, no plugin system, and no key bindings to memorize. Instead, it offers a blank surface where all commands are ordinary text that you execute with a mouse click. Any text, anywhere on screen, can be a command. Any text can be a filename. Any text can be a search term. The entire interface is built from three mouse buttons and text.

1.1 Philosophy

Rob Pike described acme's design philosophy in his 1994 paper:

> The right model is to have a shell that does not distinguish between a program that is an editor and a program that is a compiler. All programs should have equal access to the services of the environment.

This leads to several principles:

Text is the universal interface.
Commands are text. Output is text. Filenames are text. You interact with everything by reading and clicking on text.

The mouse is primary.
Acme is designed for a three-button mouse. The keyboard is for typing text; the mouse is for everything else—executing commands, opening files, selecting, searching, scrolling, and window management.

Simplicity over features.
Acme has no syntax highlighting, no auto-completion, no bracket matching, no built-in debugger, no project manager. It provides a powerful surface and lets external programs do the work.

Compose, don't configure.
Rather than configuring the editor, you compose it with external tools. Need to sort lines? Select them and execute `|sort`. Need to reformat? Execute `|fmt`. The Unix toolkit is your extension mechanism.

1.2 How Acme Differs

If you come from **vi** or **Vim**, you will notice that acme has no modes. There is no insert mode, no command mode, no visual mode. You are always editing. Commands are not key sequences—they are text you click on.

If you come from **Emacs**, you will notice that there are no key bindings to learn beyond basic cursor movement. There is no Lisp configuration, no major modes, no minor modes. The mouse does what key chords do in Emacs.

If you come from **VS Code** or similar IDEs, you will notice the absence of everything: no file tree, no tabs, no status bar, no integrated terminal widget. Instead, every window is a general-purpose text buffer. A directory listing is a window. A shell session is a window. Compiler output is a window. They are all the same thing.

The learning curve is not in memorizing commands—there are very few. The learning curve is in *changing how you think* about interacting with a computer.

1.3 History

Acme was written by Rob Pike at Bell Labs as part of the Plan 9 operating system. Plan 9 was the successor to Unix, designed by many of the same people. In Plan 9, all resources—files, network connections, processes, graphics—appear as files in a hierarchical namespace. Acme takes full advantage of this: it presents its windows, buffers, and events as a file system that other programs can read and write.

In 2003, Russ Cox ported the Plan 9 user-space tools to Unix systems as *plan9port* (Plan 9 from User Space). This made acme available on Linux, macOS, and other Unix systems, though it still depended on several Plan 9 support programs (devdraw, fontsrv, plumber, 9pserve).

The standalone version documented in this book goes further: it embeds all those support programs directly into a single binary. It uses SDL3 for graphics (supporting both X11 and Wayland on Linux, and native windows on macOS and Windows), FreeType and fontconfig for fonts, and a built-in plumber for opening files and URLs. The result is a single-binary acme with no external dependencies beyond standard system libraries.

1.4 About This Book

This book is a comprehensive guide to using acme. It covers installation on Linux, macOS, and Windows; the mouse and keyboard interface; all built-in commands; the sam editing language; plumbing; and practical workflows.

Chapter 4 (The Mouse) is the most important chapter in this book. If you read only one chapter, read that one. Acme is a mouse-driven editor, and understanding the three buttons and their chords is essential.

Chapter 8 (The Edit Command) covers the most powerful feature—a complete structural editing language inherited from the sam editor. It rewards study but is not required for day-to-day use.

The appendices provide quick-reference tables for commands, mouse actions, keyboard shortcuts, and the Unicode composition system.

Throughout this book, mouse buttons are referred to as B1 (left), B2 (middle), and B3 (right). Keyboard shortcuts are written as Ctrl+A (hold Control, press A). Built-in commands appear as `Put`, `Get`, `New`, etc.

CHAPTER 2

Installation

Acme builds from source on Linux, macOS, and Windows. The build produces a single binary, `bin/acme`, with no runtime configuration files required.

2.1 Linux

2.1.1 Prerequisites

You need a C compiler (GCC or Clang), GNU make, and the development libraries for SDL3, FreeType, fontconfig, and zlib.

Debian / Ubuntu:

```
sudo apt install gcc make
sudo apt install libsdl3-dev libfontconfig1-dev \
                 libfreetype-dev zlib1g-dev
```

Fedora:

```
sudo dnf install gcc make
sudo dnf install SDL3-devel fontconfig-devel \
                 freetype-devel zlib-devel
```

Arch Linux:

```
sudo pacman -S gcc make sdl3 fontconfig freetype2 zlib
```

Note: SDL3 is relatively new. If your distribution does not yet package it, you may need to build SDL3 from source or use a PPA.

2.1.2 Building

```
git clone <repository-url> acme
cd acme
make
```

The build compiles 18 static libraries and the acme binary. The result is `bin/acme`.

2.1.3 Running

```
./bin/acme
```

You can pass files to open on the command line:

```
./bin/acme main.c util.c README.md
```

To install system-wide, copy the binary:

```
sudo cp bin/acme /usr/local/bin/
```

2.2 macOS

2.2.1 Prerequisites

Install Xcode Command Line Tools and Homebrew packages:

```
xcode-select --install
brew install sdl3 fontconfig freetype pkg-config
```

2.2.2 Building and Running

```
make
./bin/acme
```

The build uses `pkg-config` to locate Homebrew libraries. If `pkg-config` is not available, it falls back to default Homebrew paths (`/opt/homebrew` on Apple Silicon, `/usr/local` on Intel Macs).

2.2.3 Mouse on macOS

Most Mac users do not have a three-button mouse. Acme provides modifier-click emulation:

- Ctrl-click = B2 (middle button / execute)
- Alt/Option-click = B3 (right button / look)

A standard USB or Bluetooth three-button mouse is strongly recommended for extended use. See Chapter 4 for details.

2.3 Windows

The Windows build produces a single standalone `acme.exe` that runs on any Windows 10 or later machine with no other files required — only Windows system DLLs. MSYS2 is needed at *build* time for the toolchain and static libraries; it is *not* needed at runtime.

2.3.1 Prerequisites

1. Install MSYS2 from `https://www.msys2.org/`.
2. Open the **MSYS2 UCRT64** shell from the Start menu (the icon labeled "MSYS2 UCRT64"). Avoid the plain "MSYS2 MSYS" shell — it uses the Cygwin-derived MSYS gcc, which is not what this build targets. MINGW64 and CLANG64 shells also work; the build auto-detects the active subsystem.
3. Install dependencies:

```
pacman -Syu
pacman -S git make
pacman -S mingw-w64-ucrt-x86_64-toolchain \
          mingw-w64-ucrt-x86_64-sdl3 \
          mingw-w64-ucrt-x86_64-fontconfig \
          mingw-w64-ucrt-x86_64-freetype
```

The `toolchain` group already pulls in `pkgconf`, which provides the `pkg-config` binary. Do not install `mingw-w64-ucrt-x86_64-pkg-config` explicitly — it conflicts with `pkgconf`. Note that the SDL3 package name is lowercase (`sdl3`), unlike SDL2.

A note on `PATH`. Several common Windows installs (Strawberry Perl, Anaconda, Git for Windows) ship their own `gcc` and `pkg-config` and place them on `PATH` ahead of MSYS2. Symptoms range from confusing `pkg-config` errors (“`Can't locate Pod/Usage.pm`”) to a silent missing-header failure during compilation. If `which gcc pkg-config` doesn't resolve under your active subsystem, prepend it before building:

```
export PATH="/ucrt64/bin:/usr/bin:$PATH"
```

2.3.2 Building

From the repository root:

```
make
```

This produces `bin/acme.exe` (approximately 13 MB), statically linked with no MSYS2 or Cygwin runtime dependency. Copy `acme.exe` to any Windows 10 or later machine and run it directly — no installation, no other files required.

The default font is Consolas, included with Windows. Fonts are discovered automatically from `C:\Windows\Fonts`. The `Win` command opens an interactive `cmd.exe` session inside an acme window using the Windows ConPTY API.

See `Windows-native.md` for full details.

2.3.3 Mouse on Windows

The same modifier-click emulation is available:

- Ctrl-click = B2 (middle button)
- Alt-click = B3 (right button)

Any standard three-button USB mouse works natively.

2.4 Command-Line Options

Flag	Description
`-a`	Enable auto-indent for all windows at startup.
`-c` *N*	Start with *N* columns (default: 2).
`-f` *font*	Set the proportional (variable-width) font.
`-F` *font*	Set the fixed-width font.
`-l` *file*	Load session state from a dump file.
`-r`	Reverse scrollbar button mapping (Button 1 scrolls down).
`-W` *WxH*	Set initial window size, e.g., `-W 1200x800`.
`--fonts`	List the names of all installed fonts and exit.
`-h`, `--help`	Print a usage summary, including a full explanation of font selection, and exit.

Fonts are written `Name/Size/font`, e.g. `DejaVuSansMono/13a/font`: *Name* is a font name as listed by `acme --fonts`, *Size* is the height in pixels, and a trailing `a` turns on antialiasing. The font is chosen by, in decreasing precedence: the `-f` and `-F` options, the `$font` environment variable (which sets both fonts), and the built-in default `DejaVuSansMono/13a/font`.

If the built-in default font is not installed, acme falls back to the closest installed monospace font (as chosen by fontconfig) and prints a warning. A missing font requested explicitly with `-f`, `-F`, or `$font` is a fatal error with suggested fixes.

2.5 Environment Variables

Variable	Description
`$HOME`	User home directory. Used to find `~/.config/acme/plumbing`.
`$font`	Startup font (both proportional and fixed-width), e.g. `DejaVuSansMono/13a/font`. Overridden by `-f` and `-F`.
`$acmeshell`	Shell used to run commands. Defaults to `sh`.
`$tabstop`	Default tab width in columns. Default is 4.
`$SHELL`	Shell used by the Win command.

Variable	Description
`$NAMESPACE`	Plan 9 namespace directory. Usually auto-detected.

When acme runs an external command, it sets several variables in the child's environment:

Variable	Description
`$winid`	Numeric ID of the current window.
`$%`	Filename of the current window.
`$samfile`	Same as `$%`.

CHAPTER 3

The Acme Screen

When acme starts, you see a deceptively simple display. There are no menus, no toolbars, no status bar, no file tree. There is only text, arranged in columns and windows. Every element on screen is either text you can edit or a border you can drag.

3.1 Anatomy of the Display

The acme screen is divided into a hierarchy:

1. A **top-level tag bar** spans the full width of the screen.
2. Below it, one or more **columns** are arranged side by side.
3. Each column contains one or more **windows**, stacked vertically.
4. Each window has its own **tag bar** and **body**.

3.1.1 The Top Tag Bar

The top tag bar shows global information and commands. A typical top tag bar reads:

```
Newcol Kill Putall Dump Exit
```

These are not buttons—they are editable text. You can add your own commands by typing them into the tag bar. Middle-click (B2) on any word to execute it.

3.1.2 Columns

Columns are vertical panes that divide the screen horizontally. Each column has a thin tag bar at its top, typically containing:

```
New Cut Paste Snarf Sort Zerox Delcol
```

You can:

- Create a new column by executing `Newcol` in the top tag bar.
- Delete a column by executing `Delcol` in the column's tag bar.
- Resize columns by dragging the thin border between them with B1.

3.1.3 Windows

Each window has two parts:

The tag bar is a single line (or more, if the content wraps) at the top of the window. It contains the window's filename followed by commands. A typical tag bar reads:

```
/home/user/src/main.c Del Snarf Get Put Undo Redo | Look
```

The filename at the left is editable—you can change it to rename the file, or clear it for a scratch buffer. Everything after the vertical bar `|` is the "user area" of the tag, where you can type whatever commands you find useful.

The body occupies the rest of the window and contains the file's text. This is where you read and edit.

3.1.4 The Layout Button

At the far left of each window's tag bar is a small square. This is the *layout button*. It serves two purposes:

- Drag it with B1 to move the window within its column or to another column.
- Its appearance indicates the window's state:
  - A clean square means the window matches the file on disk.
  - A filled (dirty) square means the window has unsaved changes.

3.1.5 The Scrollbar

Along the left edge of each window body is a narrow scrollbar. It shows a small rectangle (the "thumb") indicating which portion of the file is currently visible. See Section 4.5 for details on mouse interaction with the scrollbar.

3.2 Text Everywhere

The key insight of acme's interface is that *everything is text*. The tag bars are text. The body is text. Commands are text. You interact with all of it the same way: by clicking on it with the mouse.

This means:

- You can type a command anywhere—in a tag, in a body, even in the middle of a file—and middle-click it to execute.
- You can type a filename in any window and right-click it to open that file.
- Output from commands appears as text in windows, which you can then edit, search, or use as input to other commands.

There is no distinction between "code," "output," and "interface." It is all text, and it is all interactive.

3.3 Window Management

Windows in acme are not managed by tabs or a file tree. Instead:

- Open a file by right-clicking (B3) its name.
- Create an empty window with the `New` command.
- Close a window with the `Del` command.
- Clone a window with the `Zerox` command.
- Sort windows alphabetically with the `Sort` command.
- Move windows by dragging the layout button.

To navigate between windows, you simply click in the one you want. There is no "active window" highlight or focus indicator—you click where you want to work, and that window responds.

3.4 Directory Windows

Right-clicking a directory path opens a window listing the directory's contents. Each entry in the listing is itself clickable: right-click a filename to open it, right-click a subdirectory to list it.

Directory windows update automatically when files change. They serve the role of a file browser, but they are just text in a window—you can search them, select from them, or pipe their contents through commands.

3.5 The +Errors Window

When an external command writes to standard error, the output appears in a window named `+Errors` (prefixed with the directory path). These windows are created automatically as needed. You can close them with `Del` like any other window.

Compiler errors often appear in the format `file.c:27: error...`. Right-clicking (B3) on such a line opens `file.c` at line 27—a simple but powerful way to navigate error output.

CHAPTER 4

The Mouse

This is the most important chapter in this book. Acme is a mouse-driven editor. The three mouse buttons are the primary way you interact with acme—more so than the keyboard. If you learn nothing else, learn the three buttons and the chords.

Throughout this chapter, the buttons are:

B1 Left button — **Select**

B2 Middle button — **Execute**

B3 Right button — **Look / Open**

4.1 Button 1: Select

B1 is for selecting text.

Click
: Places the cursor (an empty selection) at the click position.

Drag
: Selects a range of text. The selected text is highlighted.

Double-click
: Selects the word under the cursor. A "word" is a run of alphanumeric characters and underscores, or a run of non-whitespace characters, depending on context.

Double-click on a bracket
: If you double-click on any of `( ) [ ] { } < >`, acme selects the text enclosed by the matching pair, including nested brackets. This works for parentheses, square brackets, curly braces, and angle brackets.

Double-click on a quote
: Double-clicking on a single quote, double quote, or backtick selects the text enclosed by the matching quote character.

Clicking B1 anywhere deselects the previous selection. The selection is fundamental to acme: many commands operate on the selected text, and the selected text is what gets cut, copied, or replaced.

4.2 Button 2: Execute

B2 executes text as a command.

Click on a built-in command
: Middle-click on `Put` and acme saves the file. Middle-click on `Del` and acme closes the window. Middle-click on `Undo` and acme undoes the last change.

Click on arbitrary text
: Middle-click on any text to run it as a shell command. For example, type `date` in a tag bar, middle-click it, and the current date appears in the `+Errors` window.

Click on a blank area
: If you click in an empty area or on whitespace, acme expands the selection to the surrounding non-whitespace text and executes that. This means you do not need to carefully select a command word—just click anywhere on it.

Click on a pipe command
: Text beginning with <, >, or | has special meaning. See Chapter 7 for details.

The command executes in the directory associated with the window. For a file `/home/user/src/main.c`, commands run in `/home/user/src/`.

4.3 Button 3: Look and Open

B3 is the "smart open" button. It examines the text you click on and does the most useful thing:

Filename
: Right-click on `main.c` and acme opens it in a new window (or jumps to the existing window if it is already open).

Filename with address
: Right-click on `main.c:42` and acme opens `main.c` and jumps to line 42. The address can be a line number, a line:column pair (`main.c:42.10`), or a regular expression (`main.c:/pattern/`).

Address in current file
: Right-click on `:42` or `:/pattern/` to jump to that location in the current window.

Include file
: Right-click on `<stdio.h>` and acme searches include directories for `stdio.h` and opens it.

URL
: Right-click on `https://example.com` and acme opens it in your web browser (via `xdg-open`, `open`, or `explorer` depending on platform).

Plain text
: Right-click on any other text and acme searches forward in the current window body for that text—a literal search (not a regex). Click again to find the next occurrence.

The decision-making process is handled by the *plumber* (see Chapter 9). The built-in plumbing rules handle filenames, URLs, images, and PDFs. You can add custom rules.

4.4 Mouse Chords

Mouse chords are combinations of buttons pressed in sequence. They provide quick access to cut, paste, and copy without reaching for menu commands.

4.4.1 Cut: B1–B2

1. Press and hold B1, drag to select text.
2. While still holding B1, click B2.
3. Release both buttons.

The selected text is deleted and placed in the snarf buffer (clipboard). This is identical to executing the `Cut` command.

4.4.2 Paste: B1–B3

1. Press and hold B1 to position the cursor or select text.
2. While still holding B1, click B3.
3. Release both buttons.

The snarf buffer contents replace the selection (or are inserted at the cursor position). This is identical to executing the `Paste` command.

4.4.3 Snarf: B1–B2 then B3

1. Press and hold B1, drag to select text.
2. While holding B1, click B2 (this cuts).
3. While still holding B1, click B3 (this pastes back what was just cut).
4. Release all buttons.

The net effect: the text remains unchanged, but it has been copied to the snarf buffer. This is the chord for "copy without deleting."

4.4.4 The 2-1 Argument Chord

Commands can receive an argument via a two-button chord:

1. Press and hold B2 on a command word (e.g., `Look`).
2. While holding B2, click B1 on the argument text.
3. Release both buttons.

Acme passes the B1-selected text as an argument to the B2 command. For example, holding B2 on `Look` and clicking B1 on a word searches for that word.

4.5 The Scrollbar

The scrollbar is the narrow strip along the left edge of each window body. It contains a thumb (small rectangle) showing the current viewport position relative to the whole file.

B1 in scrollbar
: Scrolls **up**. The line at the click position moves to the top of the window.

B2 in scrollbar
: **Jump scroll**. The display jumps so that the thumb is at the click position—proportional scrolling.

B3 in scrollbar
: Scrolls **down**. The line at the top of the window moves to the click position.

The `-r` command-line flag reverses the sense of B1 and B3 in the scrollbar, for users who prefer the opposite convention.

4.6 Scroll Wheel

The scroll wheel scrolls the window body:

- Scroll up = scroll text up (view earlier content).
- Scroll down = scroll text down (view later content).

The scroll amount is a fraction of the visible window height.

4.7 One-Button and Two-Button Mice

On laptops, trackpads, and Mac mice, you may not have three distinct buttons. Acme provides modifier-click emulation:

Modifier Click	Equivalent
Ctrl-click	B2 (execute)
Alt-click (or Option-click on Mac)	B3 (look/open)

These modifiers work with B1 (left click). So on a trackpad:

- Plain click = B1 (select)
- Ctrl+click = B2 (execute)
- Alt+click = B3 (look/open)

Recommendation: For serious use, get a three-button mouse. Acme was designed for one, and the experience is dramatically better with real buttons. Any inexpensive USB mouse with a clickable scroll wheel provides three buttons.

4.8 Summary

Button	Name	Action
B1	Select	Select text, position cursor, drag to move windows
B2	Execute	Run built-in commands or shell commands
B3	Look	Open files, search text, follow addresses
B1–B2	Cut	Delete selection to snarf buffer
B1–B3	Paste	Replace selection with snarf buffer
B1–B2+B3	Snarf	Copy selection to snarf buffer

CHAPTER 5

Keyboard

While the mouse is acme's primary interaction tool, the keyboard handles text entry, cursor movement, and a small set of editing shortcuts. This standalone version of acme remaps the cursor keys, Home, End, and Page keys to work in a standard, familiar way.

5.1 Cursor Movement

Key	Action
Left Arrow	Move cursor one character left.
Right Arrow	Move cursor one character right.
Up Arrow	Move cursor up one line.
Down Arrow	Move cursor down one line.
Ctrl+Left	Move to beginning of line (same as Ctrl+A).
Ctrl+Right	Move to end of line (same as Ctrl+E).
Ctrl+A	Move to beginning of line.
Ctrl+E	Move to end of line.

5.1.1 Sticky Column

The up and down arrow keys maintain a *sticky column* position. When you press Up or Down, acme remembers the horizontal pixel position of the cursor and tries to place it at the same horizontal position on the

adjacent line. If the adjacent line is shorter, the cursor goes to the end of that line, but the original column is remembered. When you reach a longer line again, the cursor returns to the remembered column.

The sticky column resets whenever you press any key other than Up or Down, or when you click the mouse.

5.1.2 Scrolling with Arrow Keys

If the cursor is on the *last visible line* and you press Down, acme scrolls down one line and places the cursor on the new last visible line. Similarly, pressing Up on the first visible line scrolls up one line. This gives smooth, continuous scrolling while navigating with arrow keys.

5.2 Page Navigation

Key	Action
Page Up	Scroll up one full screen.
Page Down	Scroll down one full screen.
Ctrl+Up	Scroll up one full screen (same as Page Up).
Ctrl+Down	Scroll down one full screen (same as Page Down).

Page Down does nothing if the end of the file is already visible.

5.3 Jump to Position

Key	Action
Home	Move cursor to first character visible on screen.
End	Move cursor to beginning of last visible line.
Ctrl+Home	Jump to beginning of file.
Ctrl+End	Jump to end of file.
Ctrl+Page Up	Jump to beginning of file (same as Ctrl+Home).
Ctrl+Page Down	Jump to end of file (same as Ctrl+End).

Home and End do *not* scroll the text—they move the cursor within the currently visible portion. Ctrl+Home, Ctrl+End, Ctrl+Page Up, and Ctrl+Page Down scroll as needed to show the target position.

5.4 Editing Keys

Key	Action
Backspace	Delete the character to the left of the cursor.
Delete	Delete the character to the right of the cursor.
Ctrl+H	Delete character left (same as Backspace).
Ctrl+W	Delete word to the left of the cursor.
Ctrl+U	Delete from cursor to beginning of line.

Backspace deletes the character to the left, which is the standard behavior on modern systems. Delete (forward delete) removes the character to the right.

5.5 Text Entry

Typing inserts text at the cursor position, replacing any selection. This is standard behavior—there are no modes.

Auto-indent. When auto-indent is enabled (via the `-a` flag or the `Indent` command), pressing Enter inserts a newline followed by the same leading whitespace as the previous line.

Tab. The Tab key inserts a literal tab character. The display width of a tab is controlled by the `Tab` command or the `$tabstop` environment variable (default: 4 columns).

5.6 Unicode Composition

Acme supports entering Unicode characters through a multi-key composition system. To compose a character:

1. Press and release the Alt key (this toggles composition mode).

2. Type the composition sequence (one or more characters).
3. The composed character is inserted.

If the sequence is unrecognized, the literal characters are inserted. Press Alt again to cancel composition mode.

5.6.1 Common Compositions

Sequence	Result	Description
`'e`	é	Acute accent
`` `e ``	è	Grave accent
`^e`	ê	Circumflex
`"e`	ë	Diaeresis / umlaut
`~n`	ñ	Tilde
`,c`	ç	Cedilla
`oe`	œ	Ligature
`*a`	α	Greek alpha
`*b`	β	Greek beta
`*p`	π	Greek pi
`!=`	$\neq$	Not equal
`<=`	$\leq$	Less or equal
`>=`	$\geq$	Greater or equal
`+-`	$\pm$	Plus-minus
`->`	$\rightarrow$	Right arrow
`<-`	$\leftarrow$	Left arrow
`sr`	$\surd$	Square root
`if`	∞	Infinity

The full composition table contains over 500 entries, covering Latin accented characters, Greek, Cyrillic, mathematical symbols, currency symbols, arrows, and more. See Appendix E for the complete table.

5.7 Summary

Category	Keys	Notes
Movement	Arrows, Ctrl+Arrows	Sticky column on up/down
Pages	PgUp, PgDn, Ctrl+Up/Dn	Full-screen scroll
Jump	Home, End	Within visible area
File jump	Ctrl+Home, Ctrl+End	Beginning / end of file
Delete	Backspace, Delete	Left / right of cursor
Words	Ctrl+W	Delete word left
Lines	Ctrl+U, Ctrl+A, Ctrl+E	Delete/move to line boundary
Compose	Alt, then sequence	Unicode character entry

CHAPTER 6

Built-in Commands

Acme's built-in commands are capitalized words that you execute by middle-clicking (B2) on them. They typically appear in window tag bars but can be typed and executed anywhere.

6.1 Clipboard: Cut, Snarf, Paste

Acme maintains an internal buffer called the *snarf buffer*. This is also synchronized with the system clipboard via SDL3, so you can copy and paste between acme and other applications.

`Cut` Delete the selected text and place it in the snarf buffer. The window is marked dirty.

`Snarf`
Copy the selected text to the snarf buffer *without* deleting it. The window is not marked dirty.

`Paste`
Replace the current selection with the contents of the snarf buffer. If there is no selection, insert at the cursor position.

The mouse chords B1–B2 (Cut), B1–B3 (Paste), and B1–B2+B3 (Snarf) perform the same operations more quickly. See Section 4.4.

To paste text from another application into acme: copy in the other application (e.g., Ctrl+C), then execute `Paste` in acme or use the B1–B3 chord.

6.2 Undo and Redo

`Undo`
: Undo the last change or group of changes. Acme groups related changes (e.g., all characters typed before a pause) into a single undo step.

`Redo`
: Reverse the last `Undo`.

Undo operates on a per-file basis. If the same file is open in multiple windows, undoing in one window affects all windows showing that file.

6.3 File Operations: Get, Put, Putall

`Get` Reload the file from disk, replacing the window's contents. If the window has unsaved changes, acme warns you (execute `Get` a second time to override).

With an argument (`Get` *`filename`*), load the named file into the current window without changing the window's name.

`Put` Write the window's contents to disk. Uses the filename shown in the tag bar.

With an argument (`Put` *`filename`*), write to the named file instead ("Save As").

`Putall`
: Write all dirty windows to their respective files. Only saves windows whose names correspond to real files. Skips scratch windows, directory windows, and windows with no filename.

6.4 Window Management

`New` Create a new empty window. With arguments (`New` *`file1 file2...`*), open those files in new windows.

`Del` Close the current window. If the window has unsaved changes, acme warns you. Execute `Del` a second time to close anyway.

`Delete`
Close the current window without checking for unsaved changes.

`Zerox`
Clone the current window, creating a second view of the same file. Both windows show the same underlying file; changes in one appear in the other.

`Newcol`
Create a new column.

`Delcol`
Delete the current column and all its windows. Warns if any window has unsaved changes.

`Sort`
Sort the windows in the current column alphabetically by filename.

6.5 Session Management: Dump, Load, Exit

`Dump`
Save the complete acme session state—window layout, filenames, font settings, and positions—to a file. Default: `$HOME/acme.dump`.

With an argument (`Dump` *`filename`*), save to the named file.

`Load`
Restore a previously saved session from a dump file. Default: `$HOME/acme.dump`.

`Exit`
Quit acme. If any window has unsaved changes, acme warns you. Execute `Exit` a second time to quit anyway.

The `Dump`/`Load` pair lets you preserve your workspace across sessions. It is common to dump before quitting and load when restarting.

6.6 Search: Look

`Look`
Search for text in the window body. Uses the current selection

as the search term. If there is no selection, uses the word nearest the cursor.

With an argument (via the 2-1 chord or by typing `Look` *`text`*), search for that text.

The search is **literal** (not a regex) and wraps around the end of the file. For regex search, use the `Edit` command (Chapter 8) or right-click on a `:/regex/` address.

6.7 Display: Font, Tab, Indent

`Font`

Toggle between proportional and fixed-width fonts.

With an argument (`Font` *`fontname`*), set the window's font. Font names are fontconfig names with a size suffix (e.g., `DejaVuSans/12a/font`).

`Tab` With no argument: print the current tab width.

With a numeric argument (`Tab 8`): set the tab display width for the current window.

`Indent`

Control auto-indentation.

- `Indent on` / `Indent off` — affect the current window.
- `Indent ON` / `Indent OFF` — affect all windows globally.

6.8 Process Control: Kill, ID

`Kill`

Send a kill signal to running commands. With arguments, kills commands whose names match.

`ID` Print the current window's numeric ID.

6.9 Include Paths: Incl

`Incl`

With no arguments: list the current supplementary include directories.

With arguments (`Incl` *`/path/to/headers`*): add directories to the search path used when right-clicking on `<filename.h>` patterns.

The default search path includes `/usr/include` and `/usr/local/include`. The `Incl` command adds project-specific directories.

6.10 Send

`Send`

In a Win window (see Chapter 7), send the text between the prompt boundary and the end of the body to the shell.

In a regular window, append the snarf buffer contents to the end of the body.

6.11 Win

The `Win` command opens an interactive shell session in an acme window. See Chapter 7 for full details.

CHAPTER 7

Shell Commands and Pipes

One of acme's most powerful features is its seamless integration with external commands. Any text can be executed as a shell command. The Unix toolkit becomes your extension mechanism.

7.1 Running External Commands

When you middle-click (B2) on text that is not a built-in command, acme runs it as a shell command. The shell used is determined by the `$acmeshell` environment variable, defaulting to `sh`.

The command runs in the directory associated with the current window. For a file `/home/user/project/main.c`, commands execute in `/home/user/project/`.

7.1.1 Environment

Acme sets the following environment variables for child processes:

- `$winid` — the numeric ID of the window.
- `$%` — the filename of the window.
- `$samfile` — same as `$%`.

7.1.2 Output

Standard output and standard error from the command appear in the `+Errors` window for that directory. If no `+Errors` window exists, one is created automatically.

7.1.3 Examples

Type any of these in a tag bar and middle-click to execute:

```
date                    # show the date
ls -la                  # list files
make                    # build the project
grep -rn TODO *.c       # search for TODOs
git status              # check version control
```

7.2 Pipe Prefixes: <, >, |

Commands prefixed with `<`, `>`, or `|` interact with the current selection:

Prefix	Behavior
`<cmd`	Run `cmd`, **replace the selection** with its standard output.
`>cmd`	Run `cmd`, sending the **selection as standard input**. The selection is unchanged.
`|cmd`	Run `cmd`, sending the **selection as standard input** and **replacing it** with standard output.

7.2.1 Examples

Replace selection with command output (`<`):

```
<date              # insert the current date
<seq 1 10          # insert numbers 1 through 10
<cat /etc/hostname # insert the hostname
```

Send selection to command (`>`):

```
>wc            # count words/lines in selection
>lp            # print selection
>xclip         # copy to X clipboard
>mail user@x   # email the selection
```

Filter selection through command (|):

```
|sort             # sort selected lines
|sort -u          # sort and remove duplicates
|fmt -w 72        # reflow text to 72 columns
|tr a-z A-Z       # uppercase the selection
|awk '{print NR, $0}'  # add line numbers
|sed 's/old/new/g'     # search and replace
|column -t        # align columns
|indent           # reformat C code
|gofmt            # reformat Go code
```

These pipe commands are the primary way to extend acme. Instead of plugins, you use the standard Unix toolkit.

7.3 The +Errors Window

Command output goes to a window named with the directory path plus `+Errors`. For example, commands run from `/home/user/project/main.c` produce output in `/home/user/project/+Errors`.

The `+Errors` window is an ordinary acme window. You can:

- Search it with B3.
- Right-click on error messages like `main.c:42: error` to jump directly to the source location.
- Close it with `Del`.
- Clear it by selecting all and deleting.

7.4 The Win Command

The `Win` command opens an interactive shell session inside an acme window. It creates a pseudo-terminal (PTY) connected to your shell (typically bash or whatever `$SHELL` is set to).

7.4.1 How It Works

1. Execute `Win` (middle-click it in any tag bar).
2. A new window opens, named after the current directory with a `-win` suffix.

3. Type commands and press Enter to send them to the shell.

4. Output from the shell appears in the window body.

7.4.2 Behavior

The Win window is not a terminal emulator—it is an acme text buffer connected to a shell via PTY. This means:

- You can edit text before pressing Enter. The shell does not see your keystrokes until you press Enter.
- You can select output text and use it like any other acme text (right-click filenames, middle-click commands, etc.).
- ANSI escape sequences are stripped. Colors and cursor positioning do not work.
- The PTY is set to single-column width, so `ls` and similar commands produce one-entry-per-line output.
- Echo is disabled—acme shows what you type, not the terminal.

7.4.3 Shell Configuration

The Win command starts the shell with a minimal environment:

- `TERM=dumb` — suppresses escape sequences.
- `PS1="$ "` — a simple prompt.
- `PROMPT_COMMAND`, `COLORTERM`, and `LS_COLORS` are unset.

The shell is started with `--noediting --norc --noprofile -i` flags (for bash) to minimize interference.

7.4.4 Closing

Close a Win window with `Del` like any other window. The shell process receives SIGHUP and terminates.

7.5 When to Use What

Need	Approach
Run a one-off command	Type it in the tag, B2 click
Transform selected text	Use `|cmd`
Insert command output	Use `<cmd`
Send text to a command	Use `>cmd`
Interactive session	Use `Win`
Navigate compiler errors	Right-click error lines in `+Errors`

CHAPTER 8

The Edit Command

The `Edit` command gives acme a complete text-processing language, inherited from Rob Pike's sam editor. It is the most powerful feature in acme and rewards study, though it is not required for daily use.

To use it, type `Edit` followed by a sam command in any tag or body, and middle-click (B2) the whole thing. For example:

```
Edit ,s/old/new/g
```

This replaces all occurrences of "old" with "new" in the current window.

8.1 Addresses

An address identifies a substring of the file. Addresses appear before commands to specify what text the command operates on.

Address	Meaning
`.`	Dot: the current selection.
`0`	The empty string at the beginning of the file.
`$`	The empty string at the end of the file.
n	Line *n*.
`#`*n*	Character position *n* (counting from 0).
`/`*regexp*`/`	Next match of *regexp* after dot.

Address	Meaning
`?`*`regexp`*`?`	Previous match of *regexp* before dot.
a`,`*b*	From address *a* to address *b*.
a`;`*b*	From *a* to *b*, with dot set to *a* before evaluating *b*.
a`+`*b*	Address *b* evaluated from the end of *a*.
a`-`*b*	Address *b* evaluated from the start of *a*, searching backward.

Examples:

- `1,5` — lines 1 through 5.
- `,` — the entire file (shorthand for `0,$`).
- `.+1` — the line after the current selection.
- `/func/` — the next occurrence of "func".
- `0,/^package/` — from start of file to the line matching "package".

8.2 Simple Commands

Command	Action
`a/`*`text`*`/`	**Append**: insert *text* after the addressed text.
`i/`*`text`*`/`	**Insert**: insert *text* before the addressed text.
`c/`*`text`*`/`	**Change**: replace the addressed text with *text*.
`d`	**Delete**: delete the addressed text.
`s/`*`regexp`*`/`*`text`*`/`	**Substitute**: replace the first match of *regexp* in the addressed text with *text*.
`s/`*`regexp`*`/`*`text`*`/g`	Substitute all matches (global flag).
`m` *`addr`*	**Move**: move the addressed text to after *addr*.
`t` *`addr`*	**Copy**: copy the addressed text to after *addr*.
`p`	**Print**: display the addressed text (in `+Errors`).
`=`	Print the line number and character offset of the address.

The delimiter for `a`, `i`, `c`, and `s` can be any character, not just `/`. Newlines within the text are entered literally (the text continues on the next line, terminated by another delimiter).

Examples:

```
Edit 1 i/// Copyright 2024\n/
Edit ,d
Edit ,s/colour/color/g
Edit 5 a/\nnew line after line 5\n/
Edit /ERROR/ p
```

8.3 Structural Commands

The structural commands are what make sam's language truly powerful. They apply commands to *each match* of a pattern, rather than to lines.

Command	Action
`x/`*`regexp`*`/`*`cmd`*	**Extract**: for each match of *regexp* in the addressed text, set dot to the match and execute *cmd*.
`y/`*`regexp`*`/`*`cmd`*	**Complement**: for each piece of text between matches of *regexp*, set dot and execute *cmd*.
`g/`*`regexp`*`/`*`cmd`*	**Guard**: execute *cmd* only if the addressed text contains a match of *regexp*.
`v/`*`regexp`*`/`*`cmd`*	**Inverse guard**: execute *cmd* only if the addressed text does *not* contain a match of *regexp*.

The key insight from Pike's "Structural Regular Expressions" paper is that `x` replaces the traditional line-oriented approach. Instead of "for each line," you say "for each match."

Idiom: "for each line" is `x/.*\n/`. Since `x` matches arbitrary patterns, it operates on structure rather than lines. You can loop over words, sentences, paragraphs, function bodies, or any textual pattern.

8.4 Grouping

Multiple commands can be grouped with braces:

```
Edit ,x/.*\n/ {
    g/TODO/ p
}
```

This prints every line containing "TODO".

8.5 Multi-File Commands

Command	Action
`X/`*`regexp`*`/`*`cmd`*	For each open file whose name matches *regexp*, execute *cmd*.
`Y/`*`regexp`*`/`*`cmd`*	For each open file whose name does *not* match *regexp*, execute *cmd*.

These operate across all open windows, not just the current one.

Example: Replace "foo" with "bar" in all open `.c` files:

```
Edit X/\.c$/ ,s/foo/bar/g
```

8.6 Substitution Details

The substitute command supports backreferences:

- `\1`, `\2`, ... refer to captured groups in the regexp.
- `&` refers to the entire match.

Example: Swap the order of two-word variable names:

```
Edit ,s/([a-z]+)_([a-z]+)/\2_\1/g
```

8.7 Worked Examples

These examples demonstrate common editing tasks using the Edit command.

8.7.1 Delete All Blank Lines

```
Edit ,x/\n\n+/ c/\n/
```

Translation: in the whole file (`,`), for each run of two or more newlines (`x/\n\n+/`), replace with a single newline (`c/\n/`).

8.7.2 Indent Selected Lines

```
Edit x/.*\n/ i/\t/
```

For each line in the selection, insert a tab at the beginning. (Select the lines first, then run this Edit command.)

8.7.3 Remove Trailing Whitespace

```
Edit ,s/[ \t]+$//g
```

8.7.4 Rename a Variable

```
Edit ,s/\boldName\b/newName/g
```

The `\b` word boundaries prevent partial matches.

8.7.5 Delete Lines Containing "DEBUG"

```
Edit ,x/.*\n/ g/DEBUG/ d
```

For each line, if it contains "DEBUG", delete it.

8.7.6 Extract Function Signatures from a C File

```
Edit ,x/^[a-zA-Z].*\n{/ p
```

Print every line that starts with a letter and ends with `{`, which is a rough match for C function definitions.

8.7.7 Number All Lines

Select the text, then:

```
Edit |awk '{printf "%4d  %s\n", NR, $0}'
```

Sometimes the easiest "Edit" command is a pipe to a Unix tool.

8.8 What Is Not Implemented

Acme implements most of the sam command language, with these exceptions:

- `k` (mark) — not implemented.
- `n` (print file list) — not implemented.
- `q` (quit) — use `Exit` instead.
- `!` (shell command) — use acme's native shell execution.

8.9 Further Reading

The Edit command language is fully described in:

- Rob Pike, "The Text Editor sam," *Software—Practice and Experience*, Vol. 17, No. 11, November 1987.
- Rob Pike, "Structural Regular Expressions," Proc. EUUG Spring 1987 Conference.
- The sam(1) manual page.

CHAPTER 9

Plumbing

When you right-click (B3) on text in acme, the *plumber* decides what to do with it. Is it a filename? Open it. A URL? Launch a browser. A compiler error? Jump to the source line. The plumber is a pattern-matching system that routes text to the appropriate action.

This standalone acme has an embedded plumber—no external daemon or configuration is needed. It comes with sensible defaults and supports custom rules.

9.1 How Plumbing Works

When you right-click on text, acme:

1. Identifies the text under the click (expands to word/filename boundaries).

2. Constructs a *plumb message* containing the text, its type, the source directory, and any attributes (like an address).

3. Matches the message against plumbing rules in order.

4. The first matching rule determines the action: open in the editor, start an external program, or search.

9.2 Default Rules

The built-in rules handle common patterns:

Pattern	Action
URLs (`http://`, `https://`, etc.)	Open in web browser.
Image files (`.jpg`, `.png`, `.gif`, etc.)	Open in image viewer.
Document files (`.pdf`, `.ps`, `.dvi`)	Open in document viewer.
`file:line.col,line.col`	Open file at line and column range.
`file:line.col`	Open file at line and column.
`file:line` or `file:/regexp/`	Open file at address.
Plain filenames	Open file in acme.
`<file.h>` style includes	Search `/usr/include` and `/usr/local/include`.

9.2.1 Platform Differences

The command used to open URLs, images, and PDFs varies by platform:

- **Linux:** `xdg-open`
- **macOS:** `open`
- **Windows:** `explorer` (dispatches through `ShellExecute`)

9.3 Custom Rules

To override or extend the default rules, create the file `~/.config/acme/plumbing`, creating the directory first if necessary:

```
mkdir -p ~/.config/acme
```

If this file exists, it *replaces* the built-in defaults entirely.

A rules file can read another file with an `include` line. Give the full path, because a relative name is looked up in the directory acme was started from:

```
include /home/me/.config/acme/go-rules
```

9.3.1 Rule Syntax

A plumbing rule is a sequence of patterns and actions, separated by blank lines. Each rule consists of:

```
# Optional comment
type is text
data matches '<regexp>'
arg isfile $1
plumb to <port>
plumb start <command> $file
```

Pattern lines test properties of the plumb message:

Pattern	Meaning
`type is text`	The message type is "text".
`data matches '`*re*`'`	The message data matches the regex. Capturing groups set `$1`, `$2`, etc.
`data set $file`	Set the data to the matched file path.
`arg isfile $`*n*	Check that the captured group is an existing file path. Sets `$file` if so.
`attr add addr=$`*n*	Add an attribute (e.g., a line number address).

Action lines specify what to do:

Action	Meaning
`plumb to edit`	Open in acme editor.
`plumb to web`	Open as a URL.
`plumb to image`	Open as an image.
`plumb client $editor`	Open in the acme editor (for "edit" port).
`plumb start` *cmd* `$file`	Run an external command.

9.3.2 Variables

Variable	Meaning
`$0`	The entire matched data.
`$1`, `$2`, …	Captured groups from `data matches`.

Variable	Meaning
`$file`	The resolved file path (set by `arg isfile`).
`$editor`	The editor variable (set at top of rules).

9.3.3 Example Custom Rules

```
# Go import paths -> go doc
type is text
data matches '[a-zA-Z0-9./\-]+'
data matches '([a-z]+\.[a-z]+/[a-zA-Z0-9./\-]+)'
plumb to web
plumb start xdg-open https://pkg.go.dev/$1
```

```
type is text
data matches 'File "([^"]+)", line ([0-9]+)'
arg isfile $1
data set $file
attr add addr=$2
plumb to edit
plumb client $editor
```

```
type is text
data matches '([a-zA-Z0-9_\-]+)\(([0-9])\)'
plumb to none
plumb start xterm -e man $2 $1
```

9.4 Debugging Plumbing

Open man pages: If right-clicking does not produce the expected result:

1. Check `~/.config/acme/plumbing` for syntax errors. If the file has a syntax error or an `include` that cannot be opened, acme reports the error at startup and runs with no plumbing rules at all.

2. Remember that custom rules *replace* the defaults—include the standard rules in your file if you want to keep them.

3. Rules are matched in order; the first match wins.

4. File existence checks (`arg isfile`) use the source window's directory for relative paths.

CHAPTER 10

Practical Workflows

This chapter demonstrates how to use acme for common programming tasks. The goal is not to prescribe a workflow but to show how acme's simple tools compose into effective patterns.

10.1 Editing Code

10.1.1 Opening a Project

Start acme with the files you want to edit:

```
acme src/*.c include/*.h Makefile
```

Or start acme empty and right-click (B3) on directory names to browse, then right-click on filenames to open them.

10.1.2 Navigating

- Right-click on a function name to search for it in the current file.
- Right-click on `header.h` to open it.
- Right-click on `:42` to jump to line 42.
- Right-click on `:/pattern/` to search by regex.
- Use `Look` with a 2-1 chord to search for specific text.

10.1.3 Multiple Columns

Use two or three columns to keep related files visible:

- Left column: header files and interfaces.
- Middle column: implementation files you are editing.
- Right column: `+Errors`, `Win` shell, or test output.

Create columns with `Newcol`. Move windows between columns by dragging the layout button.

10.2 Building and Testing

10.2.1 Running Make

Type `make` in any tag bar and middle-click it. Output appears in `+Errors`.

If the build fails with errors like:

```
main.c:42: error: undeclared identifier 'x'
```

right-click on that line to jump directly to `main.c` line 42. Fix the error, then middle-click `make` again.

10.2.2 Running Tests

```
make test
go test ./...
python -m pytest
cargo test
```

Any of these can be typed in a tag and executed. Test output with file:line references is clickable.

10.2.3 Continuous Workflow

A typical edit-build-fix cycle:

1. Edit code in a window.
2. Middle-click `Put` to save.

3. Middle-click `make` in the tag.
4. Right-click error lines to navigate to problems.
5. Fix and repeat.

For even faster iteration, open a `Win` window and run build commands interactively.

10.3 Version Control

10.3.1 Checking Status

Type in any tag and middle-click:

```
git status
git diff
git log --oneline -20
```

10.3.2 Committing

```
git add -p
git commit
```

For interactive commands like `git add -p` or `git commit` (which opens an editor), use a `Win` window.

10.3.3 Viewing Diffs

Run `git diff` and the output appears in `+Errors`. Right-click on filenames in the diff headers to open those files.

10.4 Writing Prose

10.4.1 Proportional Fonts

Switch to a proportional font for more comfortable reading:

1. Middle-click `Font` in the tag to toggle between fixed and proportional.
2. Or specify a font: `Font /mnt/font/DejaVuSerif/14a/font`

10.4.2 Reflowing Text

Select a paragraph and execute:

```
|fmt -w 72
```

This reflows the text to 72 columns, suitable for email or plain-text documents.

10.4.3 Spell Checking

Select text and run:

```
>aspell list
```

This sends the selection to `aspell` and shows misspelled words in `+Errors`.

10.5 Multi-File Search and Replace

10.5.1 Using Edit X

To replace “oldFunc” with “newFunc” in all open `.c` files:

```
Edit X/\.c$/ ,s/oldFunc/newFunc/g
```

10.5.2 Using Grep

Run `grep -rn pattern *.c` from a tag. The output lines are clickable:

```
src/main.c:42:    oldFunc(x, y);
src/util.c:17:    oldFunc(a, b);
```

Right-click each line to jump to the match, make the change, and move to the next.

10.6 Session Management

10.6.1 Saving Your Workspace

Before quitting, middle-click `Dump` in the top tag. This saves the complete state—all windows, their positions, fonts, and column layout—to `$HOME/acme.dump`.

10.6.2 Restoring

Next time you start acme:

```
acme -l $HOME/acme.dump
```

Or start acme normally and middle-click `Load` in the top tag.

10.6.3 Multiple Sessions

Save different sessions to different files:

```
Dump project-a.dump
Dump project-b.dump
```

Then load the appropriate one:

```
Load project-a.dump
```

10.7 Tips

- Add frequently-used commands to the top tag bar. For example, type `make` or `git diff` there so it is always one click away.
- Use directory windows as a file browser—right-click a directory path to list it.
- Right-clicking on a word in `+Errors` output searches for it, which is useful for tracing error messages.
- The `Zerox` command creates a second view of a file, handy for looking at one part while editing another.

- `Sort` in a column tag alphabetizes windows, making it easy to find files.

CHAPTER 11

Customization

Acme deliberately has very little to configure. There are no configuration files, no themes, no plugin system. This is by design: the power comes from composing external tools, not from configuring the editor. Nevertheless, there are several things you can adjust.

11.1 Fonts

Acme maintains two fonts: a proportional (variable-width) font and a fixed-width font. You can set both at startup and switch between them per window.

11.1.1 Startup Fonts

```
acme -f 'DejaVuSans/14a/font' -F 'DejaVuSansMono/14a/font'
```

The font name format is a fontconfig name followed by a size. The `a` suffix indicates anti-aliased rendering.

The startup fonts can also be set with the `$font` environment variable, which applies to both the proportional and fixed-width font; the `-f` and `-F` options take precedence over it:

```
export font='DejaVuSansMono/13a/font'
acme
```

If the built-in default font is not installed, acme substitutes the closest installed monospace font and prints a warning. A missing font requested explicitly with `-f`, `-F`, or `$font` is a fatal error with suggested fixes.

11.1.2 Per-Window Fonts

Middle-click `Font` in any window tag to toggle between the proportional and fixed-width font.

To set a specific font for one window, type:

```
Font /mnt/font/GoMono/13a/font
```

and middle-click the whole thing.

11.1.3 Available Fonts

Acme discovers fonts through fontconfig. Any TrueType or OpenType font installed on your system is available. To list the font names acme accepts:

```
acme --fonts
```

11.2 Tab Width

The default tab width is 4 columns. To change it:

Per window:
Middle-click `Tab 8` (or any number) in the window tag.

At startup:
Set the `$tabstop` environment variable:

```
tabstop=8 acme
```

11.3 Auto-Indentation

Auto-indent is off by default. When enabled, pressing Enter copies the leading whitespace of the current line onto the new line.

At startup:
Use the `-a` flag:

```
acme -a
```

Per window:
Middle-click `Indent on` or `Indent off` in the tag.

Globally:
Middle-click `Indent ON` or `Indent OFF` (uppercase).

11.4 Plumbing Rules

The most significant customization you can make is writing custom plumbing rules. Create `~/.config/acme/plumbing` with your rules. See Chapter 9 for the full syntax and examples.

Common customizations:

- Add rules for your programming language's error format.
- Add rules for documentation URLs (e.g., Go import paths, Rust crate docs).
- Add rules for your project's ticket system (e.g., clicking "JIRA-1234" opens the ticket in a browser).
- Change the file opener command.

11.5 Shell

11.5.1 Command Shell

The shell used for executing commands (middle-click on text) is controlled by `$acmeshell`:

```
acmeshell=bash acme
```

If not set, acme defaults to `sh`.

11.5.2 Win Shell

The `Win` command uses `$SHELL`:

```
SHELL=/bin/zsh acme
```

11.6 Helper Scripts

Since acme sets `$winid` and `$%` in the environment of child processes, you can write scripts that interact with the current window.

11.6.1 Example: Format on Save

Add to a tag:

```
Fmt
```

Where `Fmt` is a script in your PATH:

```
#!/bin/sh
# Fmt: format the current file and reload
case "$%" in
    *.go) gofmt -w "$%" ;;
    *.py) black "$%" ;;
    *.c)  indent -kr "$%" ;;
    *.rs) rustfmt "$%" ;;
esac
```

After running `Fmt`, middle-click `Get` to reload the formatted file.

11.6.2 Example: Run Tests for Current File

```
#!/bin/sh
# Test: run tests related to the current file
case "$%" in
    *_test.go) go test -run . -v "$(dirname "$%")" ;;
    *.py)      python -m pytest "$%" -v ;;
    *.c)       make test ;;
esac
```

11.6.3 Example: Open Documentation

```
#!/bin/sh
# Doc: open documentation for the word argument
word="$1"
case "$%" in
    *.go) xdg-open "https://pkg.go.dev/search?q=$word" ;;
    *.py) xdg-open "https://docs.python.org/3/search.html?q=
    $word" ;;
    *.rs) xdg-open "https://docs.rs/releases/search?query=
    $word" ;;
esac
```

11.7 Window Size

Set the initial window size with the `-W` flag:

```
acme -W 1400x900
```

11.8 Scrollbar Direction

The `-r` flag reverses the scrollbar button mapping. By default, B1 in the scrollbar scrolls up and B3 scrolls down. With `-r`, this is reversed.

```
acme -r
```

11.9 Startup Script

A common pattern is to create a shell alias or script that starts acme with your preferred settings:

```
#!/bin/sh
# ~/bin/a - start acme with my preferences
export tabstop=4
export acmeshell=bash
exec acme -a \
    -f 'GoRegular/13a/font' \
    -F 'GoMono/13a/font' \
    "$@"
```

There is no `.acmerc` or configuration file by design. A shell script gives you the same effect with full transparency.

APPENDIX A

Command Reference

All built-in commands, listed alphabetically. Commands are executed by middle-clicking (B2) on the command name.

Command	Description
`Abort`	Debug command. First execution prints a warning; second calls `abort()` to generate a core dump.
`Cut`	Delete selected text and copy to snarf buffer. Marks window dirty.
`Del`	Close window. Warns if dirty; execute again to force.
`Delcol`	Delete column and all its windows. Warns if any are dirty.
`Delete`	Close window without dirty check.
`Dump` [*file*]	Save session state. Default: `$HOME/acme.dump`.
`Edit` *cmd*	Execute sam editing command on window body. See Chapter 8.
`Exit`	Quit acme. Warns if any window is dirty; execute again to force.
`Font` [*name*]	Toggle proportional/fixed font, or set font by name.
`Get` [*file*]	Reload file from disk. With argument, load named file.
`ID`	Print window ID number.
`Incl` [*dirs...*]	List or add supplementary include directories for `<file.h>` resolution.
`Indent` [*on*|*off*|*ON*|*OFF*]	Set auto-indent. Lowercase: current window. Uppercase: all windows.

Command	Description
`Kill` [*names...*]	Send kill signal to named running commands.
`Load` [*file*]	Restore session state. Default: `$HOME/acme.dump`.
`Look` [*text*]	Literal text search in window body. Uses selection or argument.
`New` [*files...*]	Create new window. With arguments, open named files.
`Newcol`	Create a new column.
`Paste`	Replace selection with snarf buffer contents.
`Put` [*file*]	Write window to disk. With argument, write to named file.
`Putall`	Write all dirty windows whose names are real files.
`Redo`	Redo last undone change.
`Send`	In Win windows: send pending input to shell. In regular windows: append snarf buffer to body.
`Snarf`	Copy selected text to snarf buffer without deleting.
`Sort`	Sort windows in column alphabetically by name.
`Tab` [*n*]	Print or set tab width for current window.
`Undo`	Undo last change or change group.
`Win`	Open interactive shell session in new window. See Section 7.4.
`Zerox`	Clone window (second view of same file).

APPENDIX B

Edit Command Reference

This appendix summarizes the sam editing language available via acme's `Edit` command. See Chapter 8 for tutorial coverage.

B.1 Addresses

Address	Meaning
`.`	Current selection (dot).
`0`	Beginning of file.
`$`	End of file.
n	Line *n* (1-based).
`#`*n*	Character *n* (0-based).
`/`*re*`/`	Next match of regex *re* forward.
`?`*re*`?`	Next match of regex *re* backward.
a`,`*b*	Range from *a* to *b*.
a`;`*b*	Range; set dot to *a* before evaluating *b*.
a`+`*b*	*b* relative to end of *a*.
a`-`*b*	*b* relative to start of *a*, backward.
`,`	Shorthand for `0,$` (entire file).

B.2 Commands

Command	Action
`a/text/`	Append *text* after addressed range.
`c/text/`	Change: replace addressed range with *text*.
`d`	Delete addressed range.
`i/text/`	Insert *text* before addressed range.
`m addr`	Move addressed range to after *addr*.
`p`	Print addressed range.
`s/re/text/`	Substitute first match of *re*.
`s/re/text/g`	Substitute all matches.
`t addr`	Copy addressed range to after *addr*.
`=`	Print address (line number and character offset).

B.3 Structural Commands

Command	Action
`x/re/cmd`	Extract: for each match of *re*, set dot to match and run *cmd*.
`y/re/cmd`	Complement: for each stretch between matches, set dot and run *cmd*.
`g/re/cmd`	Guard: run *cmd* if addressed text matches *re*.
`v/re/cmd`	Inverse guard: run *cmd* if addressed text does *not* match *re*.
`{ cmds }`	Group multiple commands.

B.4 Multi-File Commands

Command	Action
`X/re/cmd`	For each file whose name matches *re*, run *cmd*.
`Y/re/cmd`	For each file whose name does *not* match *re*, run *cmd*.

B.5 Substitute Backreferences

In the replacement text of `s` commands:

Token	Meaning
`&`	Entire match.
`\1` ... `\9`	Captured subexpressions.
`\n`	Literal newline.
`\t`	Literal tab.
`\\`	Literal backslash.

APPENDIX C

Mouse Reference

C.1 Single-Button Actions

Button	Location	Action
B1	Body/Tag	Place cursor or select text.
B1	Scrollbar	Scroll up (line at click moves to top).
B1	Layout button	Drag to move window.
B1 double	Body/Tag	Select word.
B1 double	On bracket	Select to matching bracket.
B1 double	On quote	Select to matching quote.
B1 triple	Body/Tag	Select line.
B2	Body/Tag	Execute text as command.
B2	Scrollbar	Jump scroll (proportional).
B3	Body/Tag	Open file, follow address, or search.
B3	Scrollbar	Scroll down (top line moves to click).
Scroll up	Body	Scroll text up.
Scroll down	Body	Scroll text down.

C.2 Chords

Chord	Action
B1–B2	Cut: delete selection to snarf buffer.
B1–B3	Paste: replace selection with snarf buffer.
B1–B2+B3	Snarf: copy selection to snarf buffer (no delete).
B2 hold, B1 click	2-1 chord: pass B1 text as argument to B2 command.

C.3 Modifier-Click Emulation

For mice with fewer than three buttons:

Modifier Click	Equivalent
Ctrl+B1	B2 (execute)
Alt+B1	B3 (look/open)

C.4 Button 3 Decision Table

What happens when you right-click (B3) on different kinds of text:

Text	Action
`filename`	Open file (or jump to existing window).
`filename:N`	Open file at line N.
`filename:N.M`	Open file at line N, column M.
`filename:N.M,N.M`	Open file at line/column range.
`filename:/re/`	Open file, search for regex.
`:N`	Jump to line N in current file.
`:/re/`	Search for regex in current file.
`http://...`	Open URL in browser.
`<file.h>`	Search include directories.
Other text	Literal search in current body.

APPENDIX D

Keyboard Reference

D.1 Cursor Movement

Key	Action
Left Arrow	One character left.
Right Arrow	One character right.
Up Arrow	One line up (sticky column; scrolls at edge).
Down Arrow	One line down (sticky column; scrolls at edge).
Ctrl+Left	Beginning of line.
Ctrl+Right	End of line.
Ctrl+A	Beginning of line.
Ctrl+E	End of line.

D.2 Page and File Navigation

Key	Action
Page Up	Scroll up one screen.
Page Down	Scroll down one screen.
Ctrl+Up	Scroll up one screen.
Ctrl+Down	Scroll down one screen.

Key	Action
Home	Cursor to first visible character.
End	Cursor to start of last visible line.
Ctrl+Home	Beginning of file.
Ctrl+End	End of file.
Ctrl+Page Up	Beginning of file.
Ctrl+Page Down	End of file.

D.3 Editing

Key	Action
Backspace	Delete character left.
Delete	Delete character right.
Ctrl+H	Delete character left (same as Backspace).
Ctrl+W	Delete word left.
Ctrl+U	Delete to beginning of line.
Enter	Insert newline (auto-indent if enabled).
Tab	Insert literal tab.
Escape	Select recent typed text (cut on second press).

D.4 Composition

Key	Action
Alt (press and release)	Toggle composition mode.
Then type sequence	Insert composed Unicode character.
Alt again	Cancel composition.

See Appendix E for composition sequences.

APPENDIX E

Unicode Composition Table

To compose a character, press and release Alt to enter composition mode, then type the sequence shown. The composed character is inserted. Press Alt again to cancel composition.

The sequences below are organized by category. The full table contains over 500 entries; the most commonly useful are listed here.

E.1 Latin Accented Characters

Sequence	Char	Description
`'A`	Á	A acute
`'E`	É	E acute
`'I`	Í	I acute
`'O`	Ó	O acute
`'U`	Ú	U acute
`'a`	á	a acute
`'e`	é	e acute
`'i`	í	i acute
`'o`	ó	o acute
`'u`	ú	u acute
`` `A ``	À	A grave
`` `E ``	È	E grave

Sequence	Char	Description
`` `a ``	à	a grave
`` `e ``	è	e grave
`` `o ``	ò	o grave
`` `u ``	ù	u grave
`^A`	Â	A circumflex
`^E`	Ê	E circumflex
`^a`	â	a circumflex
`^e`	ê	e circumflex
`^o`	ô	o circumflex
`^u`	û	u circumflex
`"A`	Ä	A diaeresis
`"O`	Ö	O diaeresis
`"U`	Ü	U diaeresis
`"a`	ä	a diaeresis
`"o`	ö	o diaeresis
`"u`	ü	u diaeresis
`~N`	Ñ	N tilde
`~n`	ñ	n tilde
`~A`	Ã	A tilde
`~a`	ã	a tilde
`~O`	Õ	O tilde
`~o`	õ	o tilde
`,C`	Ç	C cedilla
`,c`	ç	c cedilla
`ss`	ß	sharp s (eszett)

E.2 Punctuation and Symbols

Sequence	Char	Description
`!!`	¡	Inverted exclamation
`??`	¿	Inverted question
`<<`	«	Left guillemet
`>>`	»	Right guillemet
`oo`	°	Degree sign

Sequence	Char	Description
`+-`	$\pm$	Plus-minus
`xx`	$\times$	Multiplication
`-:`	$\div$	Division
`co`	©	Copyright
`ro`	®	Registered

E.3 Greek Letters

Upper	Lower	Name
`*A` $\Rightarrow$ A	`*a` $\Rightarrow \alpha$	Alpha
`*B` $\Rightarrow$ B	`*b` $\Rightarrow \beta$	Beta
`*G` $\Rightarrow \Gamma$	`*g` $\Rightarrow \gamma$	Gamma
`*D` $\Rightarrow \Delta$	`*d` $\Rightarrow \delta$	Delta
`*E` $\Rightarrow$ E	`*e` $\Rightarrow \epsilon$	Epsilon
`*Z` $\Rightarrow$ Z	`*z` $\Rightarrow \zeta$	Zeta
`*Y` $\Rightarrow$ H	`*y` $\Rightarrow \eta$	Eta
`*H` $\Rightarrow \Theta$	`*h` $\Rightarrow \theta$	Theta
`*I` $\Rightarrow$ I	`*i` $\Rightarrow \iota$	Iota
`*K` $\Rightarrow$ K	`*k` $\Rightarrow \kappa$	Kappa
`*L` $\Rightarrow \Lambda$	`*l` $\Rightarrow \lambda$	Lambda
`*M` $\Rightarrow$ M	`*m` $\Rightarrow \mu$	Mu
`*N` $\Rightarrow$ N	`*n` $\Rightarrow \nu$	Nu
`*C` $\Rightarrow \Xi$	`*c` $\Rightarrow \xi$	Xi
`*P` $\Rightarrow \Pi$	`*p` $\Rightarrow \pi$	Pi
`*R` $\Rightarrow$ P	`*r` $\Rightarrow \rho$	Rho
`*S` $\Rightarrow \Sigma$	`*s` $\Rightarrow \sigma$	Sigma
`*T` $\Rightarrow$ T	`*t` $\Rightarrow \tau$	Tau
`*U` $\Rightarrow \Upsilon$	`*u` $\Rightarrow \upsilon$	Upsilon
`*F` $\Rightarrow \Phi$	`*f` $\Rightarrow \phi$	Phi
`*X` $\Rightarrow$ X	`*x` $\Rightarrow \chi$	Chi
`*Q` $\Rightarrow \Psi$	`*q` $\Rightarrow \psi$	Psi
`*W` $\Rightarrow \Omega$	`*w` $\Rightarrow \omega$	Omega

E.4 Mathematical Symbols

Sequence	Char	Description
`!=`	$\neq$	Not equal
`==`	$\equiv$	Identical / equivalent
`<=`	$\leq$	Less than or equal
`>=`	$\geq$	Greater than or equal
`<<`	$\ll$	Much less than
`>>`	$\gg$	Much greater than
`sr`	$\surd$	Square root
`if`	∞	Infinity
`+-`	$\pm$	Plus-minus
`no`	$\neg$	Logical not
`an`	$\wedge$	Logical and
`or`	$\vee$	Logical or
`el`	$\in$	Element of
`mo`	$\ni$	Contains as member
`sb`	$\subset$	Subset
`sp`	$\supset$	Superset
`ca`	$\cap$	Intersection
`cu`	$\cup$	Union
`fa`	$\forall$	For all
`te`	$\exists$	There exists
`pd`	∂	Partial derivative
`nb`	∇	Nabla / gradient
`es`	$\emptyset$	Empty set

E.5 Arrows

Sequence	Char	Description
`<-`	$\leftarrow$	Left arrow
`->`	$\rightarrow$	Right arrow
`ua`	$\uparrow$	Up arrow
`da`	$\downarrow$	Down arrow
`ab`	$\leftrightarrow$	Left-right arrow
`=>`	$\Rightarrow$	Double right arrow

E.6 Currency

Sequence	Char	Description
`eu`	€	Euro sign
`l-`	£	Pound sterling
`y=`	¥	Yen
`c|`	¢	Cent

The complete table of over 500 compositions is in the file `lib/keyboard` in the acme source distribution.

APPENDIX F

Default Plumbing Rules

The following are the built-in plumbing rules compiled into acme. They are used when `~/.config/acme/plumbing` does not exist. If you create a custom plumbing file, copy these rules as a starting point.

The opener command varies by platform: `xdg-open` on Linux, `open` on macOS, `explorer` on native Windows (which dispatches through `ShellExecute`). In the rules below, `OPENER` represents the platform-appropriate command.

F.1 Variable Definitions

```
addrelem = '((#?[0-9]+)|(/[A-Za-z0-9_\^]+/?)|[.$])'
addr     = :($addrelem([,;+\-]$addrelem)*)
twocolonaddr = ([0-9]+)[:.] ([0-9]+)
editor   = acme
```

The `addr` pattern matches sam-style addresses like `:42`, `:/pattern/`, `:$`, etc. The `twocolonaddr` pattern matches `line:col` or `line.col` pairs.

F.2 URLs

```
type is text
data matches '(https?|ftp|file|gopher|mailto|news
    |nntp|telnet|wais|prospero)://...'
```

```
plumb to web
plumb start OPENER $0
```

Matches HTTP, HTTPS, FTP, and other URL schemes. Opens in the default web browser.

F.3 Image Files

```
type is text
data matches '[a-zA-Z0-9_\-./@]+'
data matches '(...)\.(jpe?g|JPE?G|gif|GIF|tiff?
    |TIFF?|ppm|bit|png|PNG)'
arg isfile  $0
plumb to image
plumb start OPENER $file
```

Opens image files with the system image viewer.

F.4 PDF / PostScript / DVI Files

```
type is text
data matches '[a-zA-Z0-9_\-./@]+'
data matches '(...)\.(ps|PS|eps|EPS|pdf|PDF|dvi|DVI)'
arg isfile  $0
plumb to postscript
plumb start OPENER $file
```

Opens document files with the system document viewer.

F.5 File Addresses with Two-Part Line:Column

```
# file:line.col,line.col
type is text
data matches '(filename_pattern)':$twocolonaddr,$twocolonaddr
arg isfile     $1
data set       $file
attr add       addr=$2-#0+#$3-#1,$4-#0+#$5-#1
plumb to edit
plumb client $editor
```

Handles patterns like `main.c:10.5,20.3` (range with line.column on both ends).

F.6 File Addresses with Line:Column

```
# file:line.col
type is text
data matches '(filename_pattern)':$twocolonaddr
arg isfile     $1
data set       $file
attr add       addr=$2-#0+#$3-#1
plumb to edit
plumb client $editor
```

Handles patterns like `main.c:42.10` (single line.column).

F.7 Existing Files with Optional Address

```
type is text
data matches '(filename_pattern)'($addr)?
arg isfile  $1
data set    $file
attr add    addr=$3
plumb to edit
plumb client $editor
```

The catch-all file rule. Matches plain filenames and filenames with sam-style address suffixes (`main.c:42`, `main.c:/re/`).

F.8 Include Files

```
# .h files in /usr/include
type is text
data matches '([a-zA-Z0-9/_\-]+\.h)'($addr)?
arg isfile  /usr/include/$1
data set    $file
attr add    addr=$3
plumb to edit
plumb client $editor
```

```
# .h files in /usr/local/include
type is text
data matches '([a-zA-Z0-9/_\-]+\.h)'($addr)?
arg isfile  /usr/local/include/$1
data set    $file
attr add    addr=$3
plumb to edit
plumb client $editor
```

Two rules search standard system include directories. Additional directories can be added with the `Incl` command or in custom plumbing rules.

Bibliography

Pike, Rob. "Acme: A User Interface for Programmers."
Proceedings of the Winter 1994 USENIX Conference, San Francisco, 1994.
https://research.swtch.com/acme.pdf
The foundational paper describing acme's design, philosophy, and interface. Essential reading for understanding why acme works the way it does.

Pike, Rob. "The Text Editor sam."
Software—Practice and Experience, Vol. 17, No. 11, pp. 813–845, November 1987.
https://doc.cat-v.org/plan_9/4th_edition/papers/sam/
The full paper on sam's design and command language, which acme's `Edit` command implements.

Pike, Rob. "Structural Regular Expressions."
Proceedings of the EUUG Spring 1987 Conference, Helsinki, 1987.
https://doc.cat-v.org/bell_labs/structural_regexps/
Describes the theoretical foundation for sam's `x` and `y` commands—applying regular expressions to structure rather than lines.

Cox, Russ. "A Tour of Acme" (screencast).
https://research.swtch.com/acme
https://www.youtube.com/watch?v=dP1xVpMPn8M
A video demonstration of day-to-day acme use by one of its primary maintainers. The best introduction for new users.

acme(1) manual page.
https://9fans.github.io/plan9port/man/man1/acme.html
The official manual page from plan9port.

sam(1) manual page.
https://9fans.github.io/plan9port/man/man1/sam.html
Reference for the sam command language used by acme's `Edit` command.

Pike, Rob. "Plumbing and Other Utilities."
https://doc.cat-v.org/plan_9/4th_edition/papers/plumb/
Design and implementation of the plumber, Plan 9's inter-application message routing system.

Pike, Rob. "A Minimalist Global User Interface."
https://doc.cat-v.org/plan_9/4th_edition/papers/rio/
Describes rio, Plan 9's window system, which shares acme's philosophy of text and files as the universal interface.

Plan 9 from User Space (plan9port).
https://github.com/9fans/plan9port
The port of Plan 9 user-space tools to Unix, from which this standalone acme is derived.

Plan 9 Fourth Edition Papers.
https://doc.cat-v.org/plan_9/4th_edition/papers/
The complete collection of Plan 9 documentation.

Acme community resources.
https://acme.cat-v.org/
Community-maintained collection of acme tips, scripts, and links.

Index

+Errors window, 13, 32
2-1 chord, 17

acme
    introduction, 1
addresses, 35
Alt key, 22
Alt-click, 18
auto-indent, 22, 28, 53

Backspace, 22
Bell Labs, 3
body, 11

chords, 16
clipboard, 25
columns, 11
command reference, 58
command-line options, 8
commands, 25
Cox, Russ, 3
Ctrl-click, 18
cursor movement, 20
customization, 52
Cut, 25
    chord, 17

Del, 26
Delcol, 26
Delete, 26
Delete key, 22
directory windows, 13
Dump, 27, 50

Edit command, 35
    reference, 60
End key, 21
environment variables, 8
errors, 13
executing commands, 15
    external, 30
Exit, 27

Font, 28
fonts, 52

Get, 26
git, 48
grouping commands, 38

helper scripts, 55
Home key, 21

ID, 28
Incl, 28
Indent, 28
installation, 4
    Linux, 4
    macOS, 5
    Windows, 6
interactive shell, 32

keyboard, 20

reference, 65
Kill, 28

layout button, 12
Load, 27, 50
Look, 16, 27

mouse, 14
Button 1, 14
Button 2, 15
Button 3, 16
chords, 16
emulation, 18
macOS, 6
reference, 63

New, 26
Newcol, 26

Paste, 25
chord, 17
philosophy, 1
Pike, Rob, 3
pipe, prefix31
< prefix, 31
> prefix, 31
pipe commands, 30
Plan 9, 3
plan9port, 3
plumber, 41
plumbing, 16, 41
custom rules, 42
customization, 54
default rules, 42, 72
platform, 42
`~/.config/acme/plumbing`, 42
PTY, 32
Put, 26
Putall, 26

Redo, 26
sam, 35
screen layout, 10
scroll wheel, 18
scrollbar, 12, 18
direction, 56
scrolling
arrow keys, 21
page, 21
selection, 14
Send, 29
session management, 50
shell commands, 30
shell configuration, 54
Snarf, 25
chord, 17
snarf buffer, 25
Sort, 26
sticky column, 20
structural commands, 37
structural regular expressions, 35
substitute command, 38

Tab, 28
Tab key, 22
tab width, 53
tag bar
top-level, 10
window, 11

Undo, 26
Unicode composition, 22
table, 67

version control, 48

Win, 29, 32
window management, 13
window size, 56
windows, 11
workflows, 46

X command, 38

Y command, 38

Zerox, 26